what the future tests will be. Ideally these tests are small—if they need a lot of setup, you probably should be testing a smaller unit of code.

As you saw in the first few chapters, sometimes you'll pass the first test by putting in a solution that is deliberately specific to the test, like a one-line method that only returns the exact value that makes the test pass. Gary Bernhardt of Destroy All Software calls this technique *sliming*.[2] This can be helpful in keeping your code close to the tests—again, especially when the algorithm is complex.

Prescription 12	If you find yourself writing tests that already pass given the current state of the code, that often means you're writing too much code in each pass.

The goal in these tests is to first make the test pass quickly without worrying much about niceties of implementation. Once each test passes, look for opportunities to refactor. (I'll talk more about that in the next section.) When the main cases are done, you try to think of ways to break the existing code. Sometimes you'll notice something as you're writing code to pass a previous test, like, "Hey, I wonder what would happen if this argument were nil?" Write a test that describes what the output should be and make it pass. Refactoring gets increasingly important here because special cases and error conditions tend to make code complex, and managing that complexity becomes really important to future versions of the code. The advantage of waiting to do special cases at the end is that you already have tests to cover the normal cases, so you can use those to check your new code each step of the way.

When you can no longer think of a way to break your code, you're likely done with this feature and ready to move on to the next. If you haven't been doing so, run the entire test suite to make sure you didn't inadvertently break something else. Then take one further look at the code for larger-scale refactoring.

Refactoring Models

In a TDD process, much of the design takes place during the refactoring step. A lot of this design happens under the guise of cleanup—looking at parts of the code that seem overly complicated or poorly structured and figuring out how best to rearrange them.

Just because the refactoring step includes cleanup doesn't mean you can skip this step when you're in a hurry. Don't do that. Refactoring is not a luxury.

2. http://www.destroyallsoftware.com